

Project 2 Instructions

Objective: Understand branch prediction functionality and implement branch prediction related functions in the gem5 simulator. A branch predictor (BP) can be implemented by defining relevant mechanisms through following functions: `lookup`, `update`, `uncondBranch`, `btbUpdate`, and `squash`. You can find the different functions about branch predictor from <http://pages.cs.wisc.edu/~swilson/gem5-docs/classBPredUnit.html>.

1. `lookup`: This function looks up a given branch address to index appropriate registers or counters and determines if the branch needs to be taken or not taken. If global history is used, it can be updated following the prediction. The parameters for the function are `branch_address` (to look up) and `bp_history` (pointer to a new object may be set, where the object contains shared state corresponding to the lookup for this branch). The method returns the prediction, i.e., whether this branch needs to be taken or not.
2. `update`: Based on the actual taken/not taken information, this function updates the BP including counters and global history associated with the branch.
 - `branch_address`: the branch's address that will be used to index appropriate counter.
 - `taken`: whether the branch is actually taken or not
 - `bp_history`: pointer to the shared state (aka global history) corresponding to the branch
 - `squashed`: it is set to true when there is a misprediction. If so, the counter's value should not be updated. Also, the global history can be restored back if it was updated during the prediction of the branch.
3. `uncondBranch`: This method is called when the branch instruction is an unconditional branch. Branch predictor will not do anything but update the global history with taken.
4. `btbUpdate`: This function is called only when there is a branch miss in BTB. This can happen when the BP does not know where to jump. In this case, the branch is explicitly predicted as not taken so that branch target address is not required. So, any updates to the global history needs to be modified to ensure prediction as not taken.
5. `squash`: This function squashes all outstanding updates until a given sequence number. Typically, the input parameter is a `bp_history` (Pointer to the global history object associated with this branch). The predictor will need to retrieve previous state and delete the object.

Understanding 2-bit local and Bi-mode branch predictors

The 2-bit local branch predictor is a simple branch predictor which has 4 states: strongly taken, weakly taken, weakly not taken, strongly not taken, denoted as (11, 10, 01, 00). You can find the implementation in the directory at `gem5/src/cpu/pred/` in the files `2bit_local.cc`, `2bit_local.hh`, `tournament.cc` and `tournament.hh`.

Explanation about branch predictors:

- 2b local BP: <https://courses.cs.washington.edu/courses/csep548/06au/lectures/branchPred.pdf>
- bi-mode BP: <https://tnm.engin.umich.edu/wp-content/uploads/sites/353/2017/12/1997.12.Thebimode-branch-predictor.pdf>
- tournament BP: https://www.inf.ed.ac.uk/teaching/courses/car/Notes/2017-18/lecture05handling_hazards.pdf

Before starting the project, please make sure your modifications in source code or configuration of parameters made during previous projects are reverted back. If not, please correct them or obtain a new copy of gem5 from Canvas and then rebuild gem5.

Folder Setup: You will be generating a total of 2 different benchmark outputs. However, unlike the first project, you will be modifying parameters from only 1 file. You will run three separate benchmarks (dijkstra_small and qsort_small), each of which outputs the files “config.json”, “config.ini”, and “stats.txt”. At the end, you will include your code files in the submission file. We will use this for grading to ensure that your implementation works and for plagiarism checking. To generate the described file structure, run the following command as a single command (not 4 separate commands):

```
mkdir Project2-Submission Project2-Submission/Problem1 Project2-Submission/Problem1/conf1 Project2-Submission/Problem2 Project2-Submission/Problem2/conf1
```

Problem1 - Implementing Custom Branch Predictor

In this project, you will need to implement a Custom branch predictor which is very similar to the GShare branch predictor. A branch predictor GSelect that is also similar GShare is defined in the paper “*Combining Branch Predictors*” available on Canvas (Homework 3). You need to understand Sections 4-8 and Figures 6 and 8 in the paper to understand GSelect which will further help in understanding GShare and then make your own custom BP.

Task 1: Change in BranchPredictor.py

Introduce a new class CustomBP() by adding the following lines to the ‘BranchPredictor.py’ file, found in the ‘gem5/src/cpu/pred’ directory.

```
class CustomBP(BranchPredictor):
    type = 'CustomBP'
    cxx_class = 'CustomBP'
    cxx_header = "cpu/pred/custom.hh"
    PredictorSize = Param.Unsigned(1024, "Size of predictor (entries).")
    PHTCtrBits = Param.Unsigned(2, "Bits per counter.")
    globalHistoryBits = Param.Unsigned(8, "Bits of the global history.")
```

Task 2: Change in SConscript

Add the following lines at the end of the file 'SConscript' in the same directory:

```
Source('custom.cc')
DebugFlag('Mispredict')
DebugFlag('CDebug')
```

Task 3: Implement Custom Branch Predictor Functionality in Code

Create two files named custom.cc and custom.hh in the directory of `gem5/src/cpu/pred/`. You will need to define and implement the functionality for the five main branch predictor functions: `lookup`, `update`, `uncondBranch`, `btbUpdate`, `squash`, in addition to any other functions that you need to implement the Custom branch predictor. Add information about `PredictorSize`, `PHTCtrBits`, and `globalHistoryBits` into the `custom.hh` file. **As you work on your implementation, you must add meaningful and descriptive comments (at least 3 sentences) for each function to receive points. Your comments must explain what the function is doing, and you must explain major operations and logic in your code. Otherwise, you will not earn points for your code (refer to the "Rubric" section of this document for more details).**

You can look at functions in already implemented branch predictors like `2bit_local` and `bi_mode`, specifically in the `2bit_local.hh` and `bi_mode.hh` files. This will help you understand how to define relevant variables, index counters of the pattern history table for prediction, make updates after the prediction, how to work with the global history, etc.

In your `custom.hh`, variable names used for `#ifndef` and `#define` should be different from the `bi_mode.hh` or the ones used in header files of any other branch predictors.

Remember from the document: The scheme of branch predictor that you are going to make is similar to bimodal predictor or branch history table. However, like correlation-based prediction, this method records history of branches into the shift register. Then, it performs an operation (AND, OR, XOR, NOR, NAND, ...) on 'n' bits from the branch history (GR) and 'n' bits from address of the branch instruction (PC). The result is used to index the table of prediction bits. Implementation can use either tagged or tagless tables.

The operations and the no. of bits for each group will be different. The way to find out which configuration is assigned to your group is provided in the file `Find_my_configuration.pdf`

Once you make the above modifications to fully implement the Custom branch predictor, rebuild `gem5` by running the following command:

```
scons build/ARM/gem5.opt
```

Task 4: Running Benchmarks on your Implementation

Once you are confident that your implementation is correct, you will need to run the “dijkstra_small” benchmark for the each of the three different configurations (after rebuilding after each time you update the file). You don’t need to run any other benchmarks. Change the following parameters in ‘BranchPredictor.py’ to observe 2 different configurations:

Configuration	Predictor size	Counter bits	Global history bits	Benchmark
1	Configuration dependent	2	Configuration dependent	dijkstra_small
2	Configuration dependent	2	Configuration dependent	qsort_small

After modifying ‘BranchPredictor.py’ for each configuration, make sure to rebuild gem5. You can use --bp-type flag to specify the branch predictor type. In this case, we choose GSelectBP.

Run the following benchmark command (as a single line), making the appropriate substitutions:

```
./build/ARM/gem5.opt -d [path to target directory]
./configs/example/se.py --cpu-type=DerivO3CPU --caches --l2cache --bp-type=CustomBP -c ./benchmarks/dijkstra/dijkstra_small -o
./benchmarks/dijkstra/input.dat
```

```
./build/ARM/gem5.opt -d [path to target directory]
./configs/example/se.py --cpu-type=DerivO3CPU --caches --l2cache --bp-type=CustomBP -c ./benchmarks/qsort/qsort_small -o
./benchmarks/qsort/input_small.dat
```

For each configuration, you must store the results of each benchmark in the appropriate directory, which is done by putting the path to the appropriate directory for the configuration as the parameter “path to target directory” in the above commands. You created these directories as the first command in the project instructions. When you are ready to submit the project, you can move custom.cc and custom.hh into the submission folder before zipping.

Submission Instructions

Submit 1 file to canvas: **LeadersASUIId_LastName_Project2-Submission.zip**. This contains the directory structure outlined in the “Project Deliverables” section. You should be zipping just the folder “Project2-Submission” and then submitting the resulting zip file entitled “LeadersASUIId_LastName_Project2-Submission.zip” where ‘LastName’ is the last name of the

group member submitting the file to Canvas. If you are using GCP, you need to download the resulting zip file to your local machine for submission.

Use the following command to obtain the correct zip file output:

```
zip LeadersASUIId_LastName-Project2-Submission.zip -r Project2-Submission
```

Ex: If my last name is XYZ. And ASU Id is 1234567890 then the command will be:

```
zip 1234567890_XYZ-Project2-Submission.zip -r Project2-Submission
```

Only one group member should make the submission through Canvas. All team members in the group will receive the same score. Please follow the naming conventions and the directory structure correctly so that the auto grader will correctly grade your assignment. Failure to do so will result in a 1-point deduction.

Project Deliverable Structure

1. The project directory structure that you build should contain the following:
 - a. A parent directory: Project2-Submission.
 - b. 2 files inside Project2-Submission: custom.cc and custom.hh.
 - c. 2 directory inside Project2-Submission: Problem1, Problem2.
 - d. Problem1 should contain 1 directory: conf1.
 - e. Problem2 should contain 1 directory: conf1.
 - f. Each of these 2 configuration directories should contain config.ini, config.json, and stats.txt, all generated by gem5 after running the benchmark. The directory structure should be as follows:

```
Project2-Submission
|
+---custom.cc
+---custom.hh
|
+---Problem1
    +---conf1
        |   +---config.ini
        |   +---config.json
        |   +---stats.txt
+---Problem2
    +---conf1
        |   +---config.ini
        |   +---config.json
        |   +---stats.txt
```

Rubric

Any naming conventions or direct instructions that are not followed will result in a deduction of 5 points.

Accuracy [7.5 points]

- a) Correct values in the stats.txt file for Problem 1 conf1. **[4.5 points]**
- b) Correct values in the stats.txt file for Problem 2 conf1. **[4.5 points]**

Code [1 points] - Your implementations of custom.cc and custom.hh should contain useful and descriptive comments for each function and variable that you implement. Also, you did not commit plagiarism by copying code from the web without appropriate attribution, or from another group. Code will be manually checked by the graders and reviewed by a plagiarism checker.